SkyPath AI | Technical Design

RevInfinity Salesforce assessment • Prepared by Rana Ahmad Altaf • 23 October 2026

1 | Data model and relationship choices

The design separates the onboarding business record, the team capacity roster, immutable first-arrival milestones, and a reportable history of every stage visit. A new Closed Won Opportunity must create one—and only one—onboarding, even if the deal is reopened and won again.

Object / relationship	Decision and reason
Onboarding__c → Opportunity (required Lookup, restrict deletion)	Keeps an independent **OwnerId** for CSM or queue assignment; a Master-Detail child has no separate OwnerId, because parent ownership and sharing control access. A unique **Opportunity_Key__c** provides database-level duplicate protection; lookup cannot be changed.
Onboarding_Stage_Visit__c → Onboarding (Master-Detail)	Each entry—including a repeat after QA failure—is a child row with entered/exited times and duration for standard reporting. The history inherits parent access and lifecycle.
CSM_Capacity__c → User (Lookup)	Admin-editable eligibility and max active load per CSM. Unique user key prevents duplicate roster entries; eligibility excludes inactive users.
Onboarding_Assignment_Lock__c	One unique **GLOBAL** row is the mutex for competing assignments; no hardcoded record IDs are used.

2 | Automation choices and alternatives

Mechanism	Where it runs / why this tool
Opportunity after-trigger → Apex service	Detect Closed Won on inserts/updates, query existing onboarding records once, create missing children in bulk. Apex is chosen over record-triggered Flow for per-row insert outcomes, shared locking, bulk load management and layered testability.
Onboarding before/after triggers → Apex	Before: enforce stage changes, first-arrival immutability and assignment capacity. After: write stage visit history and enqueue backlog notification. Central server-side rules apply regardless of LWC, API or Data Loader.
LWC + Lightning record page	The **onboardingStageTracker** component presents current stage/action/notes with history as supporting detail; standard Dynamic Forms alone would not deliver the guided experience.
Queueable + platform notification	Notify the manager Public Group asynchronously for backlog entries, while the durable queue-owned onboarding remains the source of truth. Reports/dashboard use native Salesforce reporting.

3 | Lifecycle and timing

Ordered flow: **Kickoff Scheduled → Gathering Info → Implementation → QA / Review → Go Live → Post-Launch Review**. No skips. Gathering and implementation require notes before progressing; QA may return only to Implementation with a failure reason. First-reached timestamps remain unchanged on a revisit. Per-visit records preserve elapsed time, including repeated work.

4 | Assignment algorithm, bulk safety and concurrency

Bulk entry (200 Closed Won records in one transaction): Opportunity handler receives the trigger collection, deduplicates and checks existing onboardings with collection SOQL, then prepares the missing child records and inserts as a list—not one DML/SOQL per Opportunity. A unique Opportunity key is a second line of duplicate defense; insertion errors are fed back to the source Opportunity so a committed Closed Won does not silently lack onboarding.

Assignment for 200 records: Onboarding before-insert locks the singleton GLOBAL row using **SELECT ... FOR UPDATE**, then reads the active CSM roster and grouped active workload once. For each incoming record, choose the eligible CSM with the lowest current in-memory count and available **Max_Capacity__c**, increment that count immediately, and evaluate the next record against the updated totals. Deterministic User-ID ordering breaks workload ties. This distributes one batch fairly without exceeding per-person capacity.

Parallel transactions: All workload-changing insert/reassignment/completion paths acquire the same lock before recounting loads; this prevents two concurrent transactions claiming the last slot. A missing/contended lock must not bypass capacity rules. With no free slot, assign the **CS_Onboarding_Backlog** queue (looked up by DeveloperName), then enqueue a bell notification to members of **CS_Onboarding_Managers**. System-mode roster reads are deliberately isolated; manager reassignment checks a Custom Permission.

5 | Assumptions log (decisions for business ambiguities)

Ambiguity	Decision / consequence
What counts as won twice?	Reopen/re-close reuses the original onboarding; no new child or reset of stage history.
Which go-live date?	Use Opportunity **Target_Go_Live__c**; if blank, use **CloseDate + 30 calendar days**. Name uses Account plus date.
What is an active onboarding?	All stages except **Post-Launch Review** consume CSM capacity; Go Live remains active.
Which backward transitions?	Only **QA / Review → Implementation** for failed QA; capture a mandatory reason. No arbitrary stage skips.
What counts as stage duration?	Elapsed calendar hours between entry/exit, per visit; separate first-reached date stays immutable.
Who is eligible and what if full?	Active Salesforce users with active CSM capacity records and free slots; zero capacity is permitted, never assign beyond limit. Overflow waits in a queue.
Tie and changing workload?	Tie-break by stable User-ID ordering; reducing max capacity does not evict existing owners, but blocks new over-capacity assignments.
Who picks up backlog and gets alerted?	CS managers triage queue-owned records manually; bell notification targets members of the manager Public Group. No automatic backlog-drain job is assumed.
Who configures access?	Admins assign Permission Sets, group/queue memberships and Lightning page activation after deployment; those user-specific records are not portable metadata.

6 | Release, reproducibility and evidence

Source is SFDX format under **force-app/main/default**; deploy to a fresh Developer Edition, assign **SkyPath_Onboarding_CSM** / **SkyPath_Onboarding_Manager**, initialize the idempotent GLOBAL lock, add CSM capacity rows and manager membership, activate/verify the branded **Revinfinity | Onboarding Tracker** page, and check report/dashboard folder sharing. See README for commands and manual data setup.

Observed: clean org deployed **69/69** metadata components. All six SkyPath Apex test classes passed: **29 tests reported, 0 failures, 90% org-wide coverage** (run **707g800000qgokT**). A Closed Won deal created exactly one linked onboarding; CSM capacity 2 yielded **2 user-owned + 1 backlog-queue-owned** onboarding. Repeated lock initialization retained one row. Fresh-org UI paths, manager notification delivery and parallel-transaction stress were not all revalidated.

7 | Known limitations and next week of work

Current boundary	Next-week plan
Single assignment mutex	Stress-test parallel Data Loader jobs for row-lock waits; instrument lock duration, retry/backoff at integration layer, consider queued serialized dispatcher at greater volume.
Queue processing / notifications	Operational monitoring, failure handling, alert routing, SLA aging and safe capacity-rebalancing / automatic backlog pickup after sign-off.
Security and UI scope	Expand negative tests using distinct license types and real CSM/manager personas; UAT every stage and rollback in LWC, verify page activation and reporting folder visibility.
Stage reporting	Validate custom report types and dashboard aggregation at scale; add richer KPIs, pause reasons and aging thresholds.

8 | Scaling to a second product with a different workflow

Do **not** add a second chain of product-specific if/else branches to the current fixed six-stage service. Introduce **Product/Onboarding Type** on the onboarding, metadata-driven stage templates, ordered transitions, required-note/rollback rules and SLA targets (e.g., Custom Metadata Types). Each onboarding snapshots a template version so history remains interpretable when the product workflow evolves. Keep **Onboarding_Stage_Visit__c** generic, keyed by template-stage identifier, and use product-aware report filters. Confirm with CS whether CSM capacity is shared across products or product-specific; model multiple capacity pools only after that decision. Existing single-product records continue under their original process.

9 | Optional business-day duration stretch goal

Current **Duration_Hours__c** is elapsed calendar time, not business days. Next iteration: configure Salesforce **BusinessHours** and SkyPath holidays, compute **BusinessHours.diff** between stage entry and exit in Apex (including repeated visits), and persist an additional numeric business-hour/day field for reporting. Define whether one business day means an 8-hour schedule or the configured business-hours length; test holidays, weekends, DST and open stages.